

로딩 8초→2초, 반복 개발 90분→15분 — PC·모바일 웹 아키텍처로 성능·생산성을 끌어올리는 Frontend Engineer



정다훈

Frontend Engineer · 총 5년 9개월

휴대폰

010-4346-0429

Email

dahun428@naver.com

주소

경기 구리시 인창동

병역

[군필] 육군 병장

Portfolio

github.com/dahun428-fx

요약

React·Next.js·Vue 3를 기반으로 PC·모바일 웹 플랫폼 아키텍처를 설계하고, 성능과 생산성을 측정 가능한 수치로 개선해 온 프론트엔드 개발자입니다. 글로벌 B2B 커머스에서 PHP·jQuery 레거시를 Next.js로 전환하며 페이지 접근 시간을 8초에서 2초로 줄였고, 컴포넌트·Custom Hook·Config-Driven UI로 반복 개발 공수를 90분에서 15분으로 단축했습니다.

아키텍처를 설계할 때는 고객사별 테마·기능 노출 차이를 Config와 Base-Theme로 분리하고, 공통 컴포넌트와 렌더링 계층을 분리해 팀 엔지니어들이 같은 기준으로 컴포넌트·화면을 만들도록 가이드했습니다. Playwright E2E 자동화 600여 건·Storybook 기반 컴포넌트 검증·FE AX SOP 문서화·개발 세미나 12회로 팀 전체가 같은 기준으로 코드 품질과 개발 생산성을 관리하도록 정착시켰습니다.

기획·AI·백엔드·운영 조직과 API 응답 정책·렌더링 전략·일정을 직접 조율해 왔고, 한국미스미 글로벌 B2B 커머스 경험으로 E-commerce 도메인에도 실무 기반이 있습니다.

수많은 주문이 실시간으로 오가는 배달 O2O 플랫폼에서는 웹의 응답 속도·안정성·확장성이 곧 고객 경험이 되며, 이는 제가 글로벌 커머스와 AI 서비스 제품화에서 성능·아키텍처로 풀어 온 문제와 맞닿아 있습니다. 개발 세미나·FE AX SOP로 팀 엔지니어들이 같은 기준으로 아키텍처를 만들도록 가이드해 온 경험을 살려, 쿠팡이츠 웹 제품의 품질과 개발 생산성을 함께 끌어올리는 일을 하고 싶어 지원했습니다.

1. 글로벌 B2B 커머스 Next.js 전환 및 PC 웹 성능 개선

PHP·jQuery 기반 글로벌 쇼핑몰을 Next.js·React·TypeScript 구조로 단계적으로 전환했습니다. 페이지 접근 시간을 **8초에서 2초로** 단축했고, **SSR·CSR 렌더링 전략 분리·Lazy Loading·비동기 API·리소스 최적화**를 함께 적용해 성능·SEO·다국어·모니터링 체계를 개선했습니다.

2. Config-Driven UI·Base-Theme 기반 웹 플랫폼 아키텍처 설계

고객사마다 다른 테마·문구·기능 노출 조건을 **Config-Driven UI와 Base-Theme** 계층으로 분리했습니다. 반복 컴포넌트 개발·검증 시간을 **90분에서 15분으로** 단축하고, 신규 AI 챗봇 구축 기간을 10주에서 2주로 줄일 수 있는 공통 컴포넌트 구조를 설계했습니다.

3. Playwright E2E 자동화 및 팀 개발 생산성 기준 수립

Playwright 기반 **600여 건**의 E2E 회귀 시나리오를 자동화해 반복 QA 시간을 **3시간에서 1시간으로** 줄이고, Jenkins CI/CD 배포 리드타임을 10분에서 2분으로 단축했으며, 테스트 커버리지 98% 수준을 확보했습니다(내부 측정 기준). Storybook 컴포넌트 개발·검증 절차와 FE AX SOP 문서화, 개발 세미나 12회·기술 문서 48건으로 팀이 같은 기준으로 코드 품질을 관리하도록 정착시켰습니다.

4. 대용량 데이터 대시보드·React Native 모바일 성능 최적화

Vue 3·ECharts·RealGrid 기반 대용량 데이터 대시보드에서 렌더링 생명주기 분리와 Lazy Rendering을 적용해 렌더링 시간을 **2.5초에서 1초대로** 개선했습니다. React Native 앱에서는 검색 응답 시간을 5초에서 1초로 단축했습니다.

5. AI 서비스 초기 진입·응답 속도 개선

SSE 기반 실시간 응답 처리와 Markdown 렌더링 계층 구축, React Query 캐싱·코드 스플리팅·HTTP/2 전환을 적용해 AI 답변 출력 시간을 **10초에서 4초로**, 초기 서비스 진입 시간을 30초에서 6초로 단축했습니다. 3,493명 대상 PoC의 발화 400건 검증 기준 답변 화면 정상 출력률 100%, 같은 PoC 만족도 조사 기준 사용성 4.18·서비스 만족도 4.06·완성도 4.05를 확보했습니다.

6. 크로스팀 협업 리드 및 서비스 확장 구조 구축

기획·AI·백엔드·운영 조직과 **API 응답 형식·스트리밍 방식·오류 정책**을 직접 조율했습니다. 여러 서비스의 기능 개발과 고객사별 배포 요청, API 정책 조율을 WBS·ETA로 우선순위를 나눠 동시에 진행했고, 고객사별 CI·메뉴·기능 노출 변경을 1주 내 대응 가능한 운영 구조를 구축했습니다.

총 5년 9개월

2025.07 ~ 재직중

대웅제약 AI추진팀

주요직무: Frontend Engineer / 프론트엔드 아키텍처 설계·개발 / AI 서비스 제품화 / 팀 개발 기준 수립

React·TypeScript 기반 프론트엔드 아키텍처를 설계하고 AI 서비스 제품화와 B2B 플랫폼 내재화를 주도했습니다. Config-Driven UI·Base-Theme로 공통 컴포넌트 구조를 설계했고, Playwright·Storybook·CI/CD 기반 품질·생산성 자동화와 FE AX SOP 문서화로 팀 개발 기준을 정착시켰습니다.

- Config-Driven UI·Base-Theme 기반 공통 컴포넌트 구조 설계 및 운영 (웹 아키텍처·모듈화)
- SSE 기반 실시간 LLM 응답 스트리밍·오류 가드레일 실서비스 도입 (SSE 실시간 스트리밍)
- LLM 답변 출력 10초 → 4초, 초기 진입 30초 → 6초, Lighthouse 91점
- 반복 컴포넌트 개발·검증 시간을 90분에서 15분으로 단축 (약 83%)
- Playwright 기반 600여 건 E2E 회귀 시나리오 자동화 및 Jenkins CI/CD 빌드·배포 파이프라인 구축
- AI 생성 코드 검증 기준(FE AX SOP) 문서화, 개발 세미나 12회·기술 문서 48건 (생산성·품질 가이드)
- 기획·AI·백엔드·운영 조직과 API 응답 정책·기능 노출 조건 조율, WBS·ETA 기반 일정 관리

2020.10 ~ 2025.06

내담씨앤씨 SI사업부

주요직무: 웹 프론트엔드 개발자 / PC·모바일 웹 개발 / 레거시 전환

2020년 10월 합류해 SI 웹 프로젝트 전반에서 프론트엔드 개발·유지보수 경험을 쌓았고, 이후 글로벌 B2B 커머스·데이터 플랫폼·모바일 앱 프로젝트에서 프론트엔드 아키텍처 개선과 성능 최적화를 맡았습니다. PHP·jQuery 레거시를 React·Next.js·TypeScript 구조로 단계적으로 전환하고, Vue 3·React Native를 활용해 PC 웹과 모바일 서비스를 함께 개발했습니다.

- PHP·jQuery 기반 글로벌 B2B 커머스를 React·Next.js·TypeScript 구조로 단계적 전환 (레거시 점진 전환)
- SSR·CSR 렌더링 전략 분리 및 Lazy Loading·비동기 API·리소스 최적화로 페이지 접근 시간 8초 → 2초 단축
- 상품 비교·멀티 다운로드 등 복잡한 기능을 공통 컴포넌트·Custom Hook으로 모듈화
- Vue 3·ECharts·RealGrid 기반 대용량 데이터 대시보드 Lazy Rendering 적용, 렌더링 시간 2.5초 → 1초대
- React Native 기반 iOS·Android 앱 기능 고도화 및 플랫폼별 이슈 대응, 검색 응답 5초 → 1초 단축
- Vercel 배포 자동화, Lighthouse·GA·Adobe Analytics·Datadog 기반 성능·SEO·오류 추적 체계 운영

Frontend

React · Next.js · Vue 3 · React Native · TypeScript · JavaScript(ES6+) · Redux · Recoil · TanStack Query · RxJS · Tailwind CSS · SCSS · i18next

- React·Vue 3 기반 PC·모바일 웹 SPA 서비스 설계·개발·운영 (웹 아키텍처)
- Next.js 기반 SSR·CSR 렌더링 전략 분리, Lighthouse 병목 분석·최적화
- React Native 기반 iOS·Android 앱 기능 개발 및 플랫폼별 배포·운영 이슈 대응 (모바일 앱)
- WebView 기반 모바일 환경에서 React 웹 화면 운영 및 호환성 검증 (모바일 웹)

아키텍처 / 모듈화

Config-Driven UI · Base-Theme · 공통 컴포넌트 · Custom Hook · 렌더링 계층 분리 · Storybook

- 고객사별 테마·기능 노출 차이를 Config-Driven UI·Base-Theme 계층으로 분리 (모듈화·컴포넌트화)
- 복잡한 기능을 공통 컴포넌트·Custom Hook으로 구조화해 재사용성 확보 (계층 분리)
- Storybook 중심 컴포넌트 개발·검증 절차 수립 및 팀 가이드

성능 / 품질

Playwright · Jest · ESLint · Jenkins · GitLab CI/CD · Vercel · Lighthouse · GA4 · PostHog · Datadog · Adobe Analytics

- Playwright 기반 600여 건 E2E 회귀 시나리오 자동화, 반복 QA 3시간 → 1시간 단축 (생산성 향상)
- Jenkins·GitLab CI/CD·Vercel 빌드·검증·배포 파이프라인 구축, 배포 리드타임 10분 → 2분 단축
- FE AX SOP 문서화, 개발 세미나 12회·기술 문서 48건으로 팀 코드 품질 기준 수립

실시간 / AI / 데이터 시각화

SSE · Markdown Renderer · Chart.js · ECharts · RealGrid · Firebase FCM

- SSE 단방향 스트리밍으로 LLM 응답의 중지·재시도·오류 상태와 UI 동기화 처리
- Markdown 전용 렌더링 계층으로 표·목록·링크·차트를 React 컴포넌트로 변환
- 대용량 테이블·시계열 데이터 시각화 및 Lazy Rendering 최적화

Backend · API · Node.js/BFF

Node.js(Express) · Java · Spring Boot · Spring Framework · REST API · MySQL · MyBatis · Oracle · jQuery · Thymeleaf · WebView

- 비즈36.5에서 여러 백엔드 응답을 React 프론트 요구에 맞게 집계·프록시하는 BFF REST API를 Node.js(Express)로 신규 구축 (Node.js 웹 프레임워크)
- 비동기 I/O 이점과 프론트·백엔드 TypeScript 스택 통일을 고려해 BFF 런타임으로 Node.js 선택
- MySQL 데이터 모델·Spring Boot 로직·REST API 설계 및 Web·Admin 화면 연동 경험
- jQuery·Thymeleaf와 React가 화면 단위로 공존하는 점진적 전환 구조 경험

« 대응계약 | E2E 자동화·CI/CD로 반복 QA 3시간→1시간·배포 10분→2분 단축 »

Playwright 600여 건 자동화·FE AX SOP·개발 세미나로 팀 생산성·코드 품질 기준 정착

2026년 상반기

적용 서비스 AI코치, 비즈36.5, 바이오에이지

수행 E2E 자동화, 빌드·배포 파이프라인 정비, 운영 지표 대시보드, FE 개발 기준 문서화·세미나

[문제]

서비스 수와 공통 컴포넌트가 늘면서 수동 QA·반복 UI 개발·배포 전후 확인·운영 지표 조치가 병목이 됐습니다. AI 생성 코드의 품질·보안·일관성을 검증할 팀 공유 기준도 없어, 서비스 안정 운영을 위한 자동화 절차와 가이드 체계가 필요했습니다.

[주요 실행]

- **Playwright** 기반 주요 사용자 흐름과 E2E 회귀 시나리오를 자동화 (웹 애플리케이션 성능·생산성 향상)
- **Storybook** 중심 컴포넌트 상태별 개발·검증 절차 수립 (컴포넌트화·모듈화)
- **Jenkins CI/CD** 빌드·검증·배포 파이프라인 정비
- GA4·PostHog 기반 운영 지표 대시보드 구성
- AI 생성 코드의 컨텍스트 관리·금지 패턴·보안 위험·검증 절차를 **FE AX SOP**로 문서화 (팀 아키텍처·디자인 가이드)
- 개발 세미나 12회 운영·기술 문서 48건으로 팀 학습·적용·공유 체계 정착

[성과]

- **Playwright 기반 600여 건** E2E 회귀 시나리오 자동화
- 반복 QA 시간을 **3시간에서 1시간으로** 단축
- 반복 컴포넌트 개발·검증 시간을 90분에서 15분으로 단축 (약 83%)
- 운영 데이터 확인 절차를 3단계에서 1단계로 전환
- **Jenkins CI/CD 배포 리드타임을 10분에서 2분으로** 단축
- 기획·개발 검증 리드타임을 2일에서 1일로 단축
- 테스트 커버리지 98% 수준 확보 (내부 측정 기준)

기술 Playwright, Storybook, Jest, ESLint, TypeScript, Jenkins, GitLab CI/CD, GA4, PostHog

« 대응계약 | React 점진 전환으로 9주 만에 108개 페이지·82개 화면 프론트 내재화 »

jQuery·Thymeleaf 레거시의 React 점진 전환과 공통 컴포넌트·정책 분리로 고객사 대응 구조 확보

2025.11 ~ 2026.02

수행 jQuery·Thymeleaf→React 점진적 전환 설계, 공통 컴포넌트·권한·메뉴·브랜딩 정책 분리, WebView 앱 화면 운영, 신규 기능의 REST API·서버 로직 연동, Jenkins CI/CD

[문제] 외주 중심으로 운영되던 검진 예약 서비스를 내부 개발·운영 가능한 임직원 건강 플랫폼으로 전환해야 했습니다. jQuery·Thymeleaf 화면과 Spring Boot·MySQL 구조가 기능별로 결합되어 변경 영향 범위 파악이 어려웠고, 고객사별 CI·메뉴·기능 노출 정책이 일관되지 않아 운영 대응 속도를 높이기 어려웠습니다.

[주요 실행]

- **jQuery·Thymeleaf와 React가 화면 단위로 공존**하는 점진적 전환 구조 설계 (레거시 점진 전환)
- 반복되는 화면 요소를 공통 컴포넌트로 정리하고 권한·메뉴·브랜딩·데이터 출력 정책을 공통 기준으로 분리해 고객사별 배포 대응 구조 확보 (컴포넌트화·모듈화)
- **WebView** 기반 모바일 환경에서 React 웹 화면 운영 및 호환성 검증 (모바일 웹)
- 화면에 필요한 신규 건강관리 기능을 REST API·Spring Boot 서버 로직·MySQL 데이터 모델까지 함께 개발해 프론트 요구에 맞는 응답 구조 확보 (풀스택 오너십)
- Jenkins 빌드·검증·배포 파이프라인 정비

[성과]

- **9주 내 108개 페이지·82개 화면**을 임직원 건강 플랫폼으로 전환
- 신규 건강관리 기능 3건을 화면부터 REST API·데이터 모델까지 End-to-End 개발
- 고객사별 CI·메뉴·기능 노출 변경을 **1주 내 대응 가능한 운영 구조** 확보

기술 React, TypeScript, Tailwind CSS, jQuery, Thymeleaf, WebView, iOS, Android, Java, Spring Boot, REST API, MySQL, Jenkins CI/CD

« 대응계약 | SSE 실시간 AI 챗봇 제품화 — 응답 10초→4초·진입 30초→6초 단축 »

SSE 기반 실시간 LLM 응답과 Base-Theme·Config-Driven UI 기반 모듈화 구조 설계

2025.07 ~ 2025.10

수행 프론트엔드 아키텍처 설계·개발, 기획·AI·백엔드 조직과 API 응답 정책 조율, 실시간 응답 처리, 공통 컴포넌트 기반 서비스 확장 구조 구축

[문제] 목업 수준의 AI 건강검진 챗봇을 실사용 가능한 서비스로 고도화해야 했습니다. 초기 PoC는 답변 생성 후 일괄 출력하는 구조라 체감 대기 시간이 길었고, Markdown·표·링크·차트 등 비정형 실시간 응답을 안정적으로 렌더링하는 체계와 고객사별 화면·테마·기능 노출 조건을 일관되게 관리하는 구조가 없었습니다.

[주요 실행]

- AI 개발자·백엔드와 **API 응답 형식·스트리밍 방식·오류 정책** 조율 (크로스팀 협업 리드)
- **SSE 기반 실시간 응답**, 답변 중지·재시도·오류 상태 흐름 설계 (SSE 실시간 스트리밍)
- Markdown 전용 렌더링 계층을 구현해 표·목록·링크·차트를 **React 컴포넌트로 변환** (컴포넌트화)
- XSS 필터링과 404·500·LLM 오류 가드레일 적용으로 비정형 응답 출력 안정성 확보
- 공통 화면·테마·컴포넌트를 **Base-Theme**로 분리하고 고객사별 차이를 **Config-Driven UI**로 관리 (모듈화)
- React Query 캐싱, 코드 스플리팅, 이미지 최적화, HTTP/2 전환으로 초기 진입·네트워크 병목 개선 (웹 애플리케이션 성능 향상)

[성과]

- AI 답변 출력 시간을 **10초에서 4초로** 단축
- 초기 서비스 진입 시간을 **30초에서 6초로** 단축
- Lighthouse 91점 확보
- 3,493명 대상 PoC 운영, 3,493명 대상 만족도 조사 기준 사용성 4.18·서비스 만족도 4.06·완성도 4.05 확보
- 3,493명 대상 PoC의 400건 발화 검증 기준 답변 화면 정상 출력률 100% 달성
- 신규 AI 챗봇 구축 기간을 10주에서 2주로 단축 가능한 공통 컴포넌트 구조 설계

기술 React, TypeScript, TanStack Query, Recoil, SSE, Chart.js, Tailwind CSS

« 삼성물산 | Vue 3 대용량 대시보드 2.5초→1초대·RN 검색 5초→1초 성능 개선 »

Vue 3 기반 대용량 데이터 대시보드 성능 최적화 및 React Native 모바일 서비스 고도화

2024.10 ~ 2025.04

소속 내담씨앤씨 SI 프로젝트

수행 대용량 데이터 시각화 성능 개선, Spring REST API·데이터 연동, React Native iOS·Android 개발·배포

[문제] Web 대시보드는 대용량 테이블·시계열 데이터를 빠르게 제공해야 했고, 모바일 앱은 반복 API 호출과 플랫폼별 동작 차이로 사용자 대기 시간과 운영 부담이 발생했습니다.

[주요 실행]

- **Vue 3·ECharts·RealGrid** 기반 대용량 데이터 시각화 화면 개발
- 테이블·차트 **렌더링 생명주기를 분리**하고 Lazy Rendering을 적용해 초기 처리량 최적화 (성능 개선)
- Spring REST API와 필터링·정렬 로직을 설계하고 사용자 역할별 데이터 조회·표현 구조 구성
- 검색 시마다 API를 호출하던 구조를 화면 진입 시 **비동기 선조회 후 Recoil에 저장**하는 방식으로 변경 (UX 개선)
- React Native 기반 iOS·Android 공통 기능, Firebase FCM 실시간 알림, 플랫폼별 배포·운영 이슈 대응 (모바일 앱)

[성과]

- 대용량 데이터 대시보드 렌더링 시간을 **2.5초에서 1초대로** 개선
- React Native 앱 검색 응답 시간을 **5초에서 1초로** 단축

기술 Vue 3, React Native, TypeScript, ECharts, RealGrid, Java, Spring, REST API, MyBatis, MySQL, Recoil, SQLite, Firebase FCM

« 한국미스미 | 글로벌 B2B 커머스 Next.js 전환 — 페이지 접근 8초→2초 단축 »

PHP·jQuery 레거시 분석, Next.js·React 전환, 렌더링 전략·성능·다국어·모니터링 구조 개선

2022.06 ~ 2024.10

소속 내담씨앤씨 SI 프로젝트

수행 PHP·jQuery 레거시 분석, Next.js·React·TypeScript 단계적 전환, SSR·CSR 렌더링 전략 분리, 성능·SEO·다국어·배포 자동화·모니터링 구조 개선

[문제]

PHP·jQuery 기반 글로벌 B2B 커머스 쇼핑몰은 화면 결합도가 높아 상품·주문 기능 확장과 유지보수가 어려웠습니다. 단일 렌더링 구조로 인해 초기 페이지 접근 시간이 길고 검색 노출(SEO) 최적화에 한계가 있었습니다. 다국어 환경과 국가별 배포 요건을 지원하면서 서비스를 단계적으로 현대화해야 했습니다.

[주요 실행]

- **PHP·jQuery 레거시를 Next.js·React·TypeScript** 컴포넌트 구조로 단계적 전환 (레거시 점진 전환)
- 페이지 특성별 **SSR·CSR 렌더링 전략 분리** (웹 아키텍처 설계)
- 상품 비교·멀티 다운로드 등 복잡한 기능을 공통 컴포넌트·Custom Hook으로 구조화 (컴포넌트화·모듈화)
- Redux 기반 상태 관리로 화면 간 상태 일관성 확보
- Lighthouse 병목 분석 후 **Lazy Loading, 비동기 API, 폰트·리소스 최적화** 적용 (웹 애플리케이션 성능 향상)
- i18next 기반 글로벌 다국어 구조 구축 (모바일·PC 웹 솔루션)
- Vercel 배포 자동화, **Datadog·GA·Adobe Analytics** 기반 운영 모니터링 체계 구축

[성과]

- 페이지 접근 시간을 **8초에서 2초로** 단축
- Next.js 전환 후 기존 PHP 서비스 대비 평균 로딩 속도 개선
- 화면 개발에서 아키텍처·성능·다국어·배포 자동화·모니터링까지 담당 범위 확장

기술 Next.js, React, TypeScript, JavaScript, Redux, RxJS, i18next, PHP, jQuery, Twig, Vercel, Datadog, Lighthouse, GA, Adobe Analytics

학력

2010.03 ~ 2017.02

성공회대학교 일어일본학과 졸업

복수전공: 경영학과

학점: 3.8 / 4.5

2010

여의도고등학교 졸업

교육

2020.03 ~ 2020.09 웹 콘텐츠 개발을 위한 응용SW엔지니어링
중앙에이치티에이(주)

Java 기반 웹 애플리케이션의 백엔드와 프론트엔드 개발 과정 이수

Spring Framework를 활용한 팀 프로젝트 진행

습득 기술:

HTML5, JavaScript, jQuery, Vue.js, CSS, JSP, Java, Spring Framework, AWS, RESTful API, OracleDB, MariaDB, Git/GitHub, Subversion, Tomcat, Eclipse

자격증

정보처리기사

한국산업인력공단

어학

영어

취득일: 2024.05

TOEIC 825점

일본어

취득일: 2018.08

JLPT 1급